

DATA 266 | HOMEWORK 1

Autoregressive Models, Neural Networks, and CUDA

A reproducible comparison of PyTorch, TensorFlow, and GPU matrix multiplication

Student: Nikhil Kanaparthi

SID4: 6441

Assignment: Homework 1

Submission tag: hw1

Date: September 1, 2026

Executive summary

This report evaluates autoregressive modeling concepts, compares matched feedforward neural networks in PyTorch and TensorFlow, and measures a tiled CUDA matrix-multiplication kernel against a CPU baseline. The assigned HP_ID 3 modification reduced the learning rate from 0.001 to 0.0003. The lower learning rate slowed optimization in both frameworks. TensorFlow produced more stable accuracy across seeds, while CUDA delivered increasing end-to-end speedups as matrix size grew.

1. Personal parameters

Parameter	Value	Use in HW1
SID4	6441	Student-specific identifier
SEED	6441	Fixed split and primary training seed
SLICE	441	Reported standing parameter; not otherwise used
HP_ID	3	Lower-learning-rate experimental arm
CLS_A	1	Reported standing parameter; not otherwise used
CLS_B	7	Reported standing parameter; not otherwise used

2. Autoregressive models

An autoregressive model predicts the next value from values that precede it. For a sequence x_1 through x_T , the joint probability is decomposed into a product of conditional probabilities, where each term predicts x_t from x_1 through $x_{(t-1)}$. This factorization converts sequence generation into repeated next-step prediction.

Real-world examples include language models that predict the next token, speech systems that generate the next audio sample or frame, time-series models that forecast future measurements from past observations, and image generators that produce pixels or patches in a defined order. The defining property is directional dependence on previously generated or observed elements.

3. Diabetes dataset and preprocessing

The supplied CSV has no header row. It contains 759 examples, eight normalized numeric features, and a binary outcome. Loading with `header=None` preserves the first data row; descriptive feature names are assigned explicitly. The file contains no missing values. Class 0 has 263 examples and class 1 has 496, so stratification is used to preserve the class ratio.

The data is divided once into 70% training, 15% validation, and 15% testing subsets using `random_state=6441`. This produces 531 training examples, 114 validation examples, and 114 testing examples. The same split is retained for every framework, model configuration, and training seed. Because the course file is already normalized, no second scaling pass is applied.

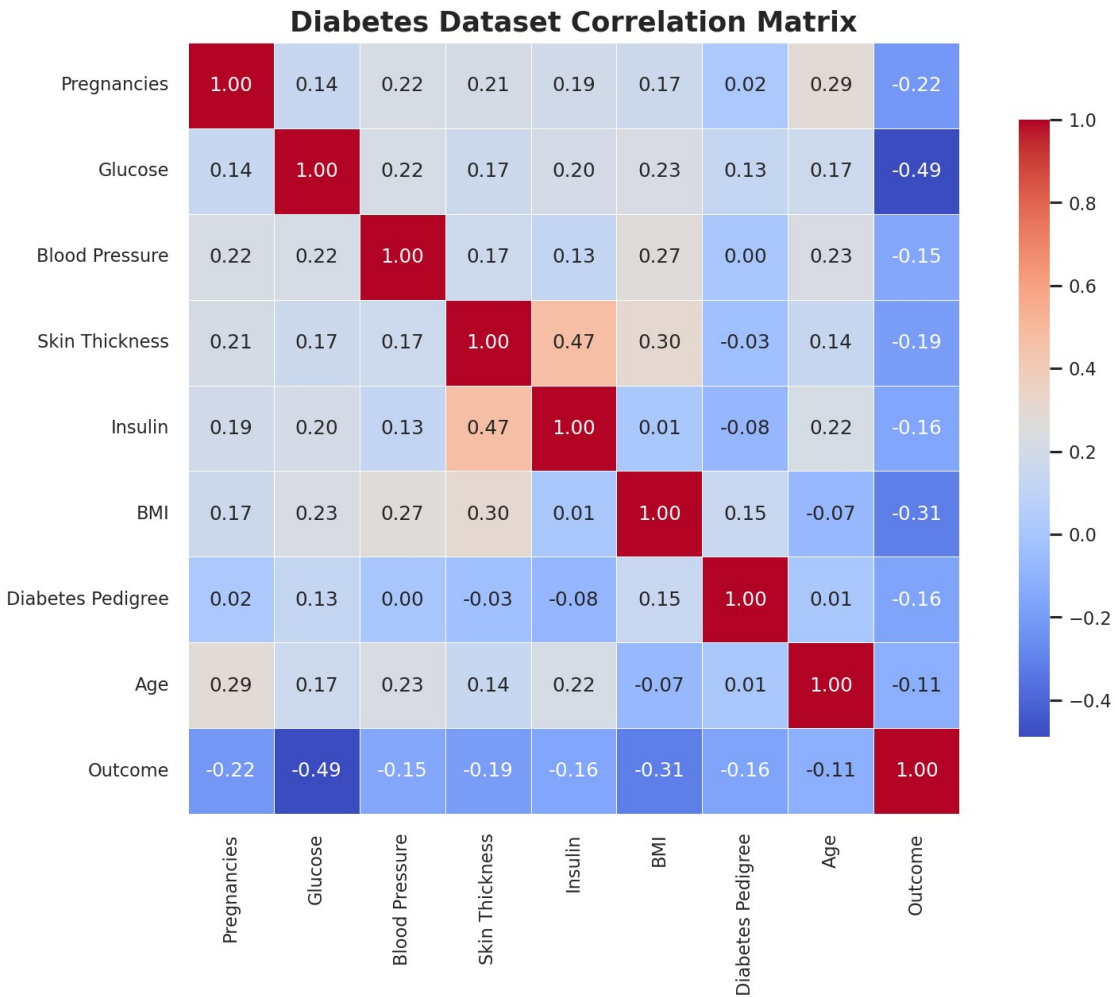


Figure 1. Correlation matrix for the supplied normalized diabetes dataset.

Normalized Diabetes Feature Distributions

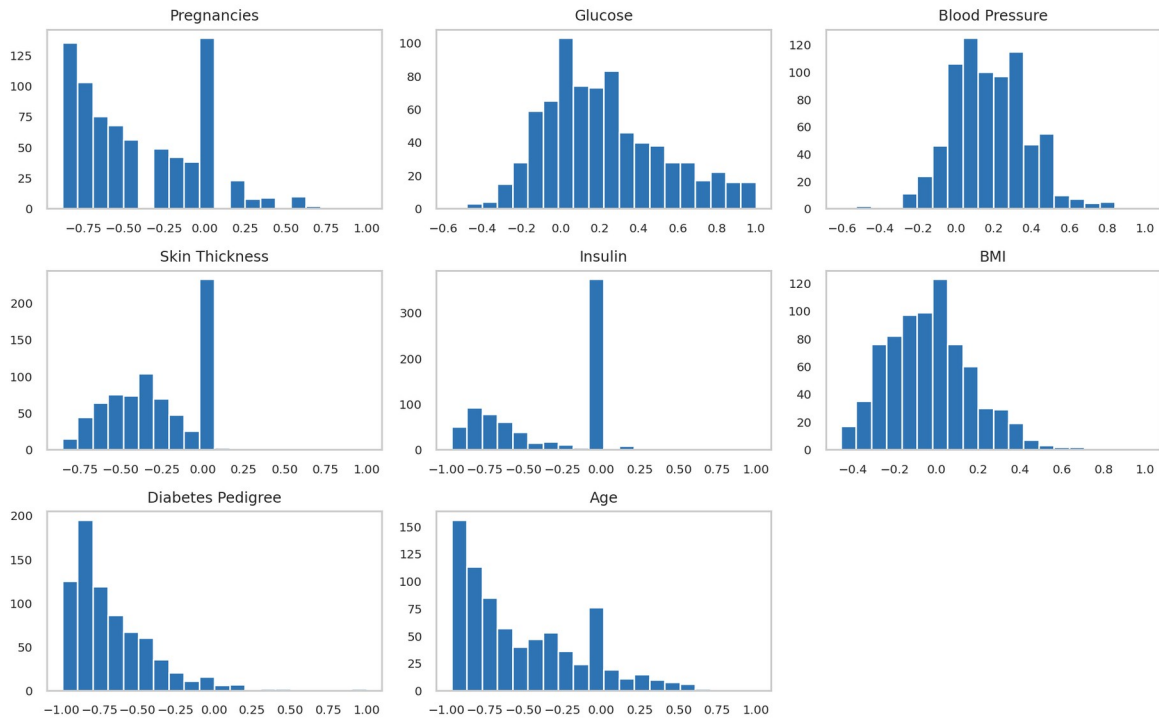


Figure 2. Distributions of the eight normalized input features.

4. Neural-network methodology

The matched baseline network in both frameworks uses eight inputs, hidden layers [64, 32] with ReLU activations, and one sigmoid output. Binary cross-entropy is the loss function, SGD is the optimizer, and batch size is 32. The baseline uses learning rate 0.001 for 30 epochs. HP_ID 3 keeps the same architecture and epoch budget but lowers the learning rate to 0.0003.

Each configuration is trained three times using seeds 6441, 6442, and 6443 while the dataset split remains fixed at random_state=6441. Test accuracy is reported for each run, followed by the sample mean and sample standard deviation. Training and validation loss curves use the seed-6441 run.

5. Neural-network results

5.1 Accuracy summary

Framework	Model	Learning rate	Mean accuracy	Std. dev.
PyTorch	Baseline	0.001	55.263%	18.232%
PyTorch	Modified HP_ID 3	0.0003	54.971%	17.984%
TensorFlow	Baseline	0.001	68.129%	3.321%
TensorFlow	Modified HP_ID 3	0.0003	62.865%	2.680%

Framework and model	Seed 6441	Seed 6442	Seed 6443
PyTorch baseline	65.789%	65.789%	34.211%
PyTorch modified	65.789%	64.912%	34.211%
TensorFlow baseline	66.667%	65.789%	71.930%
TensorFlow modified	60.526%	65.789%	62.281%

5.2 PyTorch

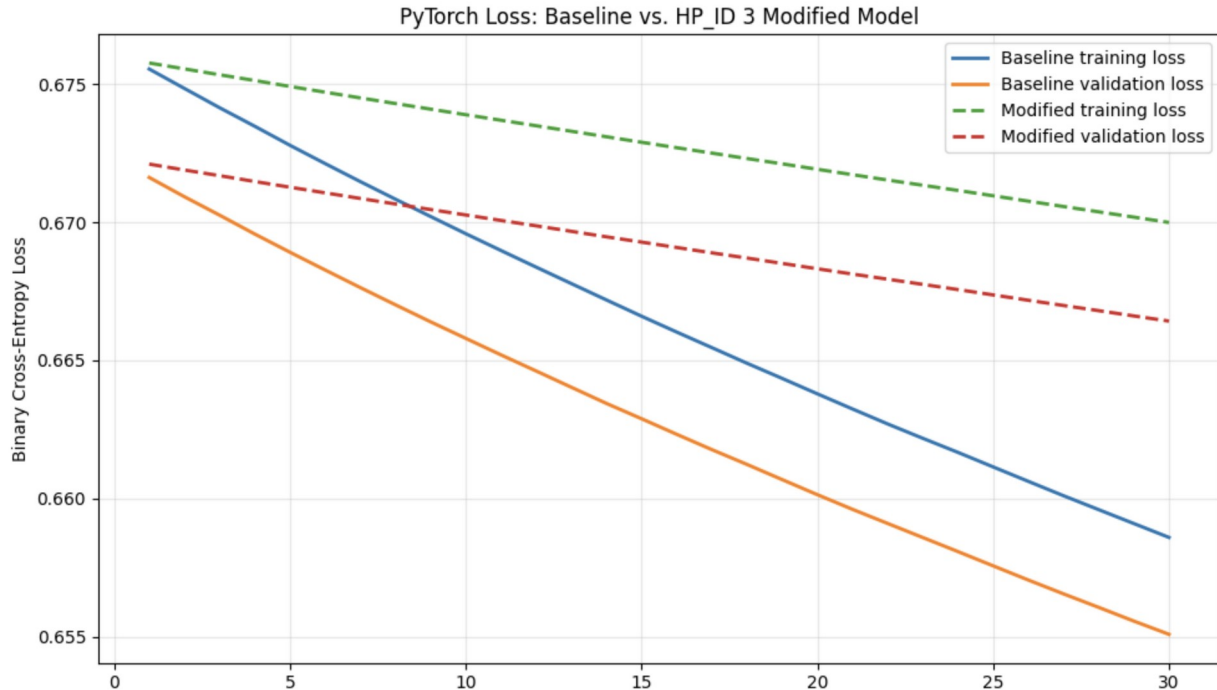


Figure 3. PyTorch training and validation loss for seed 6441.

The PyTorch baseline achieved 55.263% mean test accuracy with 18.232% standard deviation, while the modified model achieved 54.971% with 17.984% standard deviation. Their means differ by only 0.292

percentage points, far less than the approximately 18-point seed variation, so their test accuracies are indistinguishable across these runs.

The curves still show a training-behavior difference. Baseline training and validation loss fall to about 0.659 and 0.655, while modified losses end near 0.670 and 0.666. Both validation curves continue downward, so there is no evidence of overfitting. The lower learning rate instead slows optimization and leaves the modified model more underfit after 30 epochs.

5.3 TensorFlow

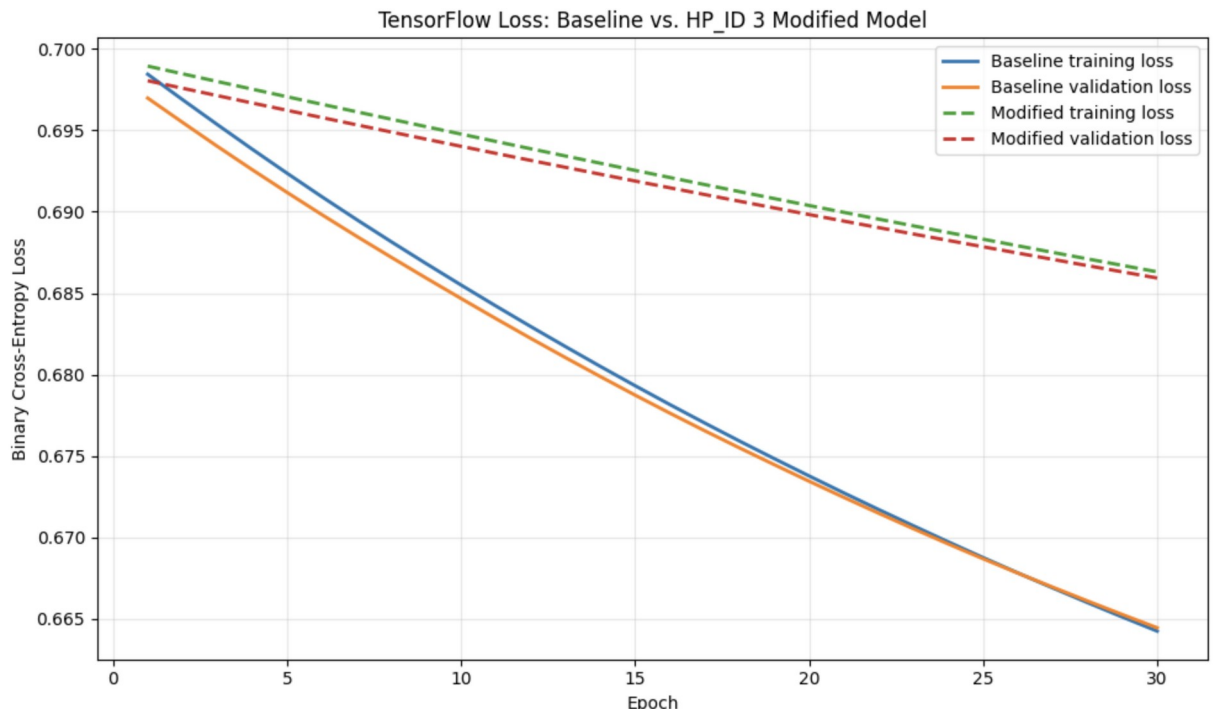


Figure 4. TensorFlow training and validation loss for seed 6441.

The TensorFlow baseline achieved 68.129% mean accuracy with 3.321% standard deviation. The modified model achieved 62.865% with 2.680% standard deviation, a decrease of 5.264 percentage points. The mean difference is larger than either run-to-run standard deviation, so the baseline appears better in this three-seed experiment.

Baseline losses fall from about 0.698 to 0.664, whereas modified losses only reach about 0.686. Training and validation losses stay close and decrease together, indicating slower learning rather than overfitting.

5.4 Framework comparison

TensorFlow is more accurate and stable in these recorded runs. Its baseline is 68.129% $\pm$ 3.321%, compared with 55.263% $\pm$ 18.232% for PyTorch. Its modified result is 62.865% $\pm$ 2.680%, compared with 54.971% $\pm$ 17.984% for PyTorch. This does not establish that one framework is generally superior; different initialization, batching, and numerical implementations can produce different outcomes even when the nominal architecture and settings match.

6. CUDA matrix multiplication

6.1 Kernel design

The CUDA kernel uses 16 x 16 two-dimensional blocks, giving 256 threads per block. Each thread computes one output element $C[\text{row}, \text{col}]$. The grid covers the complete output matrix, and boundary checks protect accesses when a dimension is not divisible by 16. Shared-memory tiles from A and B reduce repeated global-memory reads; threads synchronize before using each tile and before loading the next tile.

The CPU reference uses cache-friendlier i-k-j loop ordering. Both implementations use identical deterministic inputs, and every GPU result is checked against the CPU result with a relative-error tolerance. All required sizes passed correctness verification.

6.2 Timing methodology and results

A small CPU operation and one full GPU kernel are used as warm-ups. Each required size is then measured three times and the median is reported. GPU allocation is excluded. GPU end-to-end time is kernel time plus host-to-device and device-to-host transfer time.

Matrix size	CPU (ms)	GPU kernel (ms)	H2D+D2H (ms)	End-to-end (ms)	Speedup
256	3.161	0.113	0.256	0.369	8.578x
1024	219.810	3.257	2.964	6.221	35.345x
4096	26,208.769	200.268	44.798	245.066	106.946x

6.3 Profiler verification

The 1024 x 1024 program was profiled with nvprof. The profiler separated the tiled matrix-multiplication kernel from host-to-device and device-to-host memory copies. It recorded two kernel calls because the program includes one warm-up call and one measured call.

GPU activity	Total time	Calls	Average per call	Share
Tiled kernel	11.590 ms	2	5.795 ms	71.18%
Host to device	3.210 ms	4	0.802 ms	19.99%
Device to host	1.258 ms	2	0.629 ms	7.83%

The program's measured iteration reported 5.817 ms of kernel time and 3.136 ms of combined transfer time, consistent with the profiler's per-call breakdown. The main comparison retains the three-repeat medians because profiling adds overhead.

6.4 Crossover analysis

The smallest measured size where GPU end-to-end execution is beneficial is 256×256 . The CPU takes 3.161 ms, while the GPU takes 0.369 ms including transfers, for an 8.578x speedup. Speedup grows to 35.345x at 1024 and 106.946x at 4096 as arithmetic work grows faster than fixed launch and transfer overhead. The crossover cannot occur at size zero because kernel launch and data movement have nonzero fixed costs; sufficiently small workloads may not contain enough parallel work to recover them.

7. Conclusion

The HP_ID 3 learning-rate reduction consistently slows optimization. In PyTorch, accuracy differences are overwhelmed by seed variation, making the configurations indistinguishable on test accuracy despite lower baseline losses. In TensorFlow, the baseline is 5.264 percentage points more accurate on average and also learns faster. The CUDA experiment demonstrates the complementary systems result: parallel hardware becomes increasingly advantageous as the matrix size grows, while transfers and launch overhead explain why GPU benefit is not guaranteed for arbitrarily small problems.

Sources

Course-provided Homework 1 assignment, homework template, diabetes.csv, Demo 1 neural-network notebooks, and Week 1 CUDA demonstration notebooks. All reported numerical results come from the executed student notebooks and profiler output.